BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Ereditarietà ed interfacce

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- Ereditarietà
- Il modificatore final
- La classe Object
- Rapporto ereditarietà-incapsulamento
- Quando utilizzare l'ereditarietà
- Il modificatore abstract
- Interfacce

Ereditarietà

Nel mondo reale è possibile classificare tutto con classi e sottoclassi. Per esempio un cane è un animale, un aereo è un veicolo, la chitarra è uno strumento musicale.

Ereditarietà

In Java l’ereditarietà è la caratteristica che mette in relazione di estensibilità più classi che hanno caratteristiche comuni. Per esempio la classe Cane estenderà la classe Animale.

Ereditarietà

Grazie all’utilizzo della parola chiave **extends** è possibile implementare l’ereditarietà.

Ereditarietà

Si consideri il seguente script di codice:

```
package libreria;

public class Libro {
   public String titolo;
   public String autore;
   public String editore;
   public int numeroPagine;
   public int prezzo;
   //...
}
```

```
package libreria;

public class LibroSuJava {
   public String titolo;
   public String autore;
   public String editore;
   public int numeroPagine;
   public int prezzo;
   public final String ARGOMENTO_TRATTATO = "Java";
   //...
}
```

Ereditarietà

Le classi Libro e LibroSuJava rappresentano due concetti in relazione tra loro e quindi dichiarano campi in comune. Utilizzando il concetto di ereditarietà è possibile metterle in relazione utilizzando la seguente sintassi:

```
package libreria;

public class LibroSuJava extends Libro{

    public final String ARGOMENTO_TRATTATO = "Java";
    //...
}
```

Ereditarietà

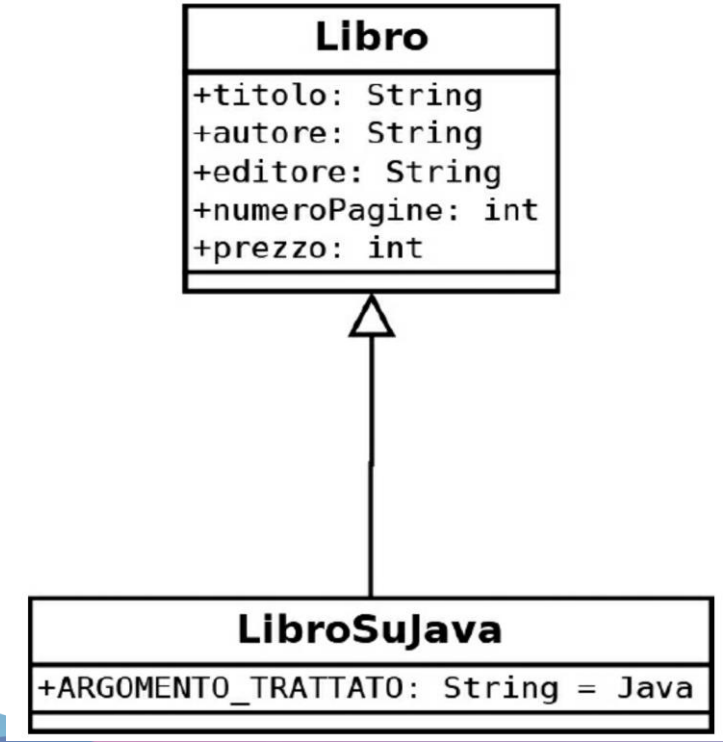
In questo modo la classe LibroSuJava erediterà tutti i campi pubblici della classe Libro, estendendola con le variabili pubbliche numeroPagine, prezzo, titolo, autore ed editore, anche se non sono state codificate esplicitamente.

Ereditarietà

La costante **ARGOMENTO_TRATTATO** invece, appartiene solo alla classe LibroSuJava. In particolare LibroSuJava è una sottoclasse di Libro, e Libro è superclasse di LibroSuJava.

Ereditarietà

Il Class Diagram “Ereditarietà: LibroSuJava è sottoclasse di Libro”.



Il modificatore final

Il modificatore **final** può essere utilizzato non solo con le variabili, ma anche con classi e metodi per influire sulle regole dell'ereditarietà e rendendole "non modificabili".

Il modificatore final

Ad esempio:

- Se dichiaro final una variabile diventa costante.
- Un metodo dichiarato final non può essere riscritto in una sottoclasse.
- Una classe dichiarata final non può essere estesa.

Il modificatore final

Un esempio di classi dichiarate final sono le classi String e la classe Math del package java.lang.

La classe Object

Nella libreria standard di Java esiste una classe chiamata Object che astrae il concetto di oggetto generico, appartiene al **package java.lang** ed è la superclasse di ogni classe.

La classe Object

Quindi, quando si crea una classe, questa viene inserita nella gerarchia di classi alla cui cima c’è Object.

Rapporto ereditarietà-incapsulamento

Si consideri da un punto di vista pratico una **classe Ricorrenza** ottenuta specializzando la **classe Data** come esempio di incapsulamento.

Rapporto ereditarietà-incapsulamento

Essendo tutti i metodi **set e get** dichiarati pubblici nella superclasse, verranno ereditati, quindi resi utilizzabili nella sottoclasse.

Rapporto ereditarietà-incapsulamento

Un errore comune è quello di pensare che per poter utilizzare una certa variabile in una sottoclasse, sia necessario dichiararla **protected**, invece è sufficiente avere a disposizione i metodi **mutator (set)** ed **accessor (get)** nelle sottoclassi.

Rapporto ereditarietà-incapsulamento

Relazione Is-a

Per poter fare un uso corretto dell'ereditarietà, il programmatore deve porsi una semplice domanda:

Rapporto ereditarietà-incapsulamento

“Un oggetto della candidata sottoclasse è un oggetto della candidata superclasse?”

Se la risposta è positiva si applica l’ereditarietà, nel caso contrario no.

Rapporto ereditarietà-incapsulamento

Ereditarietà e costruttori

L’ereditarietà non è applicabile ai costruttori.

Rapporto ereditarietà-incapsulamento

La parola chiave **super,** definita “reference implicito all’intersezione tra l’oggetto corrente e la sua superclasse”, è strettamente legata al concetto di costruttore.

Rapporto ereditarietà-incapsulamento

In ogni costruttore, infatti, è sempre presente una chiamata al costruttore della superclasse, tramite una sintassi speciale che sfrutta il reference super.

Rapporto ereditarietà-incapsulamento

Per esempio nella classe **LibroSuJava,** il costruttore verrà modificato dal compilatore nel seguente modo:

```
package libreria;

public class LibroSuJava extends Libro{

   //...

   public LibroSuJava() {
      super(); //istruzione implicita se non fornita esplicitamente.
      System.out.println("Costruito un Libro su Java!");
   }

}
```

Rapporto ereditarietà-incapsulamento

Il costruttore della classe Libro viene invocato dal costruttore della classe LibroSuJava.

Rapporto ereditarietà-incapsulamento

Se si hanno più costruttori nella superclasse, è possibile scegliere quale chiamare. Ad esempio se la classe Libro definisse più costruttori nel seguente modo:

```
package libreria;

public class Libro {
   //...
   public Libro(String titolo, String autore) {
      this(titolo); // Chiamata al secondo costruttore
      setAutore(autore);
   }
   public Libro(String titolo) {
      this.titolo = titolo;
   }
   //...
}
```

Rapporto ereditarietà-incapsulamento

La classe LibroSuJava potrebbe chiamare a seconda del caso il costruttore più adeguato.

Per esempio:

```
package libreria;

public class LibroSuJava extends Libro{

   //...

   public LibroSuJava(String titolo) {
      super(titolo);
   }
   public LibroSuJava(String titolo, String autore) {
      super(titolo, autore);
   }
   //...
}
```

Rapporto ereditarietà-incapsulamento

La chiamata tramite **super()** ad un costruttore di una superclasse o tramite **this()** ad un costruttore della classe stessa, può essere usata solo come prima istruzione in un costruttore.

Rapporto ereditarietà-incapsulamento

Se una di queste istruzioni non fosse inserita come prima istruzione, il compilatore segnalerebbe un errore. Questo implica che solo un'istruzione tra **super()** o **this()** sarà presente in un costruttore.

Rapporto ereditarietà-incapsulamento

In questo caso un costruttore chiama un altro costruttore mediante il comando **this()** che a sua volta o chiama un terzo costruttore tramite un altro comando **this(),** o chiama un costruttore della superclasse con il comando **super().**

Rapporto ereditarietà-incapsulamento

È possibile utilizzare **this** e **super** per chiamare anche metodi, non solo costruttori. Circa this si utilizza la seguente sintassi:

this.nomeMetodo()

Sintassi simile utilizzando super:

super.nomeMetodo()

Rapporto ereditarietà-incapsulamento

Si considerino le seguenti classi:

```
package persona;

public class Persona {
   private String nome, cognome;

   public String toString() {
      return nome + " " + cognome;
   }
   //accessor e mutator methods (set e get)
}
```

```
package persona;

public class Cliente extends Persona {
   private String indirizzo, telefono;

   public String toString() {
      return super.toString()+ "\n" + indirizzo + "\nTel:" + telefono;
   }
   //accessor e mutator methods (set e get)
}
```

Rapporto ereditarietà-incapsulamento

La **classe Persona** dichiara un metodo che si chiama toString(), dichiarato direttamente dalla classe Object, e dovrebbe essere il metodo responsabile a rappresentare con una stringa l'oggetto su cui viene chiamato il metodo.

Rapporto ereditarietà-incapsulamento

Quindi la classe Persona sta effettuando un **override** del metodo, facendo sì che il metodo ritorni una stringa composta dal nome e dal cognome.

Rapporto ereditarietà-incapsulamento

La classe Cliente estende la classe Persona e ne eredita anche il metodo toString(). Questo metodo viene a sua volta ridefinito per stampare anche i nuovi campi definiti nella classe Cliente: indirizzo e telefono.

Rapporto ereditarietà-incapsulamento

Per stampare anche il nome e il cognome, il metodo toString() della classe Cliente chiama il metodo toString() della superclasse Persona e poi concatena il risultato con i dettagli di indirizzo e telefono.

Rapporto ereditarietà-incapsulamento

Ereditarietà e modificatori

Il modificatore **private** può essere applicato a variabili e metodi, impedendo a tali membri di essere ereditati in una sottoclasse.

Rapporto ereditarietà-incapsulamento

Il modificatore **protected,** può essere applicato solo a variabili e metodi permettendo che i membri siano ereditati.

Rapporto ereditarietà-incapsulamento

Ricapitolando:

- Una Classe dichiarata final non può essere estesa.
- Un Metodo dichiarato final non può essere riscritto (override) in una sottoclasse.

Rapporto ereditarietà-incapsulamento

- Una Variabile final diventa una costante.

È possibile ereditare una costante (a meno che non sia dichiarata private).

Rapporto ereditarietà-incapsulamento

Il modificatore **static** può essere utilizzato con il comando **import,** per importare membri statici di una classe, ma soprattutto può essere applicato a variabili, metodi ed anche inizializzatori.

Rapporto ereditarietà-incapsulamento

Le variabili e i metodi statici, vengono ereditati, ma siccome appartengono alla classe e non agli oggetti, si tratta di un’ereditarietà tra classi.

Rapporto ereditarietà-incapsulamento

Gli inizializzatori statici e d’istanza vengono in qualche modo ereditati, ma non si tratta di ereditarietà vera e propria, semmai del normale flusso di esecuzione.

Rapporto ereditarietà-incapsulamento

A scopo illustrativo si consideri la seguente classe che definisce due semplici inizializzatori (uno statico e uno d'istanza), oltre ad un costruttore senza parametri.

Rapporto ereditarietà-incapsulamento

```
package initializer;
public class ParentInitializer {
   public ParentInitializer() {
      System.out.println("Costruttore di ParentInitializer");
   }
   {
      System.out.println("Inizializzatore d'istanza di ParentInizializer");
   }
   static {
      System.out.println("Inizializzatore statico di ParentInizializer");
   }
}
```

Rapporto ereditarietà-incapsulamento

Si estenda la classe ChildInitializer che dichiara a sua volta altri due inizializzatori e un metodo main() che la auto-istanzia:

Rapporto ereditarietà-incapsulamento

```
package initializer;

public class ChildInitializer extends ParentInitializer{
   public ChildInitializer() {
      System.out.println("Costruttore di ChildInitializer");
   }
   {
      System.out.println("Inizializzatore d'istanza di ChildInizializer");
   }
   static {
      System.out.println("Inizializzatore statico di ChildInizializer");
   }
   public static void main(String args[]) {
      new ChildInitializer();
   }
}
```

Rapporto ereditarietà-incapsulamento

OUTPUT:

Inizializzatore statico di ParentInizializer

Inizializzatore statico di ChildInizializer

Inizializzatore d'istanza di ParentInizializer

Costruttore di ParentInitializer

Inizializzatore d'istanza di ChildInizializer

Costruttore di ChildInitializer

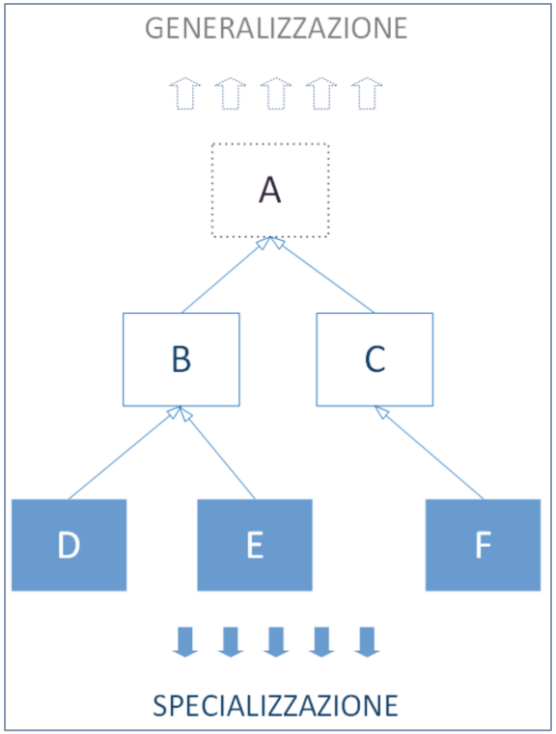
Rapporto ereditarietà-incapsulamento

La classe **ParentInitializer** deve essere caricata in memoria prima di caricare la classe **ChildInitializer**.

Rapporto ereditarietà-incapsulamento

I termini che descrivono i processi che portano all'implementazione dell'ereditarietà sono:

- **Generalizzazione**
- **Specializzazione**



Rapporto ereditarietà-incapsulamento

Si parla di **generalizzazione** se, a partire da un certo numero di classi, si definisce una superclasse che ne raccoglie le caratteristiche comuni.

Rapporto ereditarietà-incapsulamento

Invece si parla di **specializzazione** quando partendo da una classe, si definiscono una o più sottoclassi allo scopo di ottenere oggetti più specializzati.

Il modificatore abstract

Il modificatore **abstract** può essere applicato sia ai metodi che alle classi. Per definizione il modificatore non presenta parentesi graffe nella sintassi ma termina con un punto e virgola:

public abstract void dipingiQuadro();

Il modificatore abstract

Il metodo astratto può essere definito solo all’interno di una classe astratta. Una classe astratta non può, però, essere istanziata.

Il modificatore abstract

Infatti dichiarare una **classe abstrac** implica che deve essere estesa e quindi si avranno delle sottoclassi, ciò comporta che l’interazione con il software sarà più semplice se si ragiona pensando alle astrazioni piuttosto che alle implementazioni.

Il modificatore abstract

Si consideri la seguente classe astratta:

```
package pittore;

public abstract class Pittore {

   //...

   public abstract void dipingiQuadro();

   //...

}
```

Il modificatore abstract

Questa classe ha senso se inserita in un sistema in cui l'oggetto Pittore può essere considerato troppo generico per definire un nuovo tipo di dato da istanziare.

Il modificatore abstract

Si supponga che per il sistema sia fondamentale conoscere lo stile pittorico di un Pittore, ma non esistono pittori che riescono a dipingere con qualsiasi stile, converrebbe creare sottoclassi non astratte di Pittore, PittoreImpressionista e PittoreNeoRealista.

Il modificatore abstract

Queste sottoclassi devono ridefinire il metodo astratto dipingiQuadro.

Il modificatore abstract

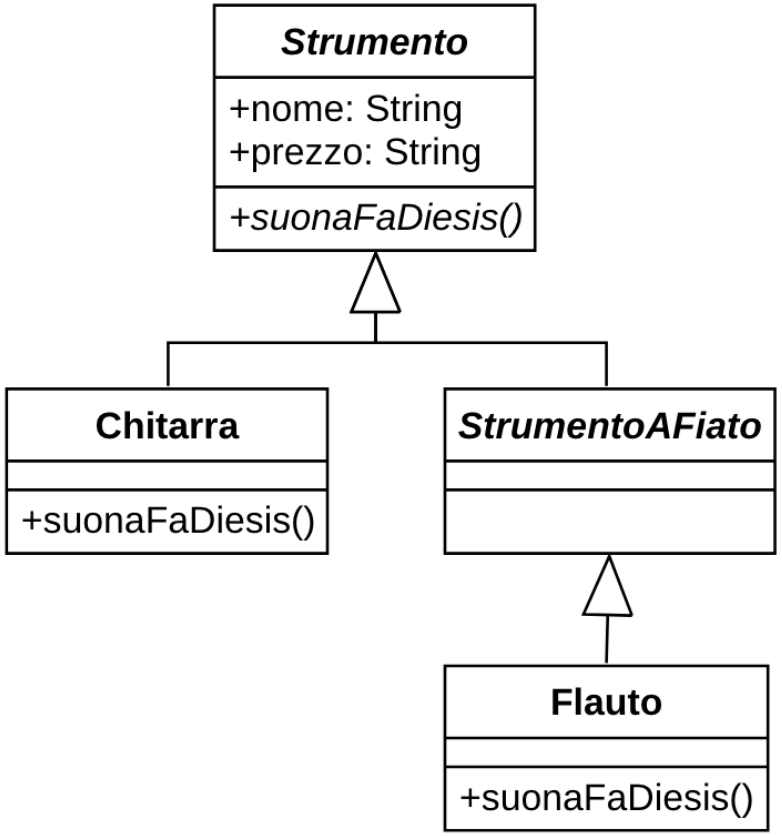
```
package pittore;

public class PittoreImpressionista extends Pittore{

	public void dipingiQuadro() {
		System.out.println("Quadro impressionista");
	}
}
```

```
package pittore;

public class PittoreNeoRealista extends Pittore{

   public void dipingiQuadro() {
      System.out.println("Quadro neo-realista");
   }
}
```

Il modificatore abstract

A scopo illustrativo si consideri la classe Strumento, considerando i vari strumenti musicali come sottoclassi.

Il modificatore abstract



Il modificatore abstract

```
package strumenti;

//Classe astratta
public abstract class Strumento {
   public String nome;
   public String prezzo;

   /* Ogni strumento suona in modo
    * diverso! Impossibile definire
    * questo metodo!
    */
   public abstract void  suonaFaDiesis();

   //...
}
```

Il modificatore abstract

```
package strumenti;

//Classe concreta
public class Chitarra extends Strumento {

   // Override (riscrittura) del metodo
   public void suonaFaDiesis() {
      //Implementazione del metodo per la chitarra.
   }
   //...

}
```

```
package strumenti;

// Classe di nuovo astratta che estende Strumento
public abstract class StrumentoAFiato extends Strumento {

   //metodo suonaFaDiesis ereditato ancora astratto e non riscritto
   //perché troppo generico!

   //...
}
```

Il modificatore abstract

```
package strumenti;

// Classe concreta che estende StrumentoAFiato
public class Flauto extends StrumentoAFiato {

   //Implementazione del metodo per il flauto.
   public void suonaFaDiesis() {

   }

   //...

}
```

Il modificatore abstract

Nell’esempio considerato Strumento è la superclasse della classe concreta Chitarra (che ridefinisce il metodo suonaFaDiesis()) e della classe astratta StrumentoAFiato.

Il modificatore abstract

Infine la classe concreta Flauto estende StrumentoAFiato (che a sua volta estendeva Strumento). Quindi la chitarra è uno strumento e il flauto è uno strumento a fiato, ma è anche uno strumento: è tutto coerente!

Il modificatore abstract

Il grande **vantaggio** che offre l'implementazione di una **classe astratta è che obbliga le sue sottoclassi ad implementare un comportamento.**

Il modificatore abstract

Il modificatore abstrac, circa le classi, può essere considerato l’opposto del modificatore final.

Il modificatore abstract

Attenzione!

In Java **non esiste** propriamente quella che viene definita **ereditarietà multipla**: una classe può estendere solo una classe per volta, anche se grazie alle interfacce è possibile.(da Java 8)

Interfacce

Dal punto di vista della progettazione, un’interfaccia è un’evoluzione del concetto di classe astratta. Dal punto di vista del codice, assomiglia invece ad una classe senza la sua implementazione interna.

Interfacce

Fino a Java 7 si avevano solo metodi dichiarati implicitamente public e abstract e variabili dichiarate implicitamente public, static e final.

Interfacce

Un’interfaccia, essendo un’evoluzione di una classe astratta, non può essere istanziata.

Per essere utilizzata, ha bisogno di essere in qualche modo estesa, ed è possibile solo da un’altra interfaccia.

Interfacce

Dopo Java 8 le interfacce hanno acquisito maggiore potenza, ma contemporaneamente le regole che ne governano l'utilizzo si sono complicate.

Interfacce

Metodi statici

È possibile:

- Definire all'interno delle interfacce anche metodi statici.
- Scrivere interfacce nel seguente modo:

```
public interface StaticMethodInterface {

    static void metodoStatico() {
        System.out.println("Metodo Statico Chiamato!");
    }
}
```

Interfacce

È importante chiamare il metodo sfruttando il nome dell’interfaccia e non il nome della sua sottoclasse.

Interfacce

Metodi di default

È possibile dichiarare metodi concreti all'interno delle interfacce. Si parla di metodi di default perché vengono dichiarati usando come modificatore la parola chiave **default.**

Interfacce

Si consideri il seguente codice:

```
package musica;

public interface Solista {

   default void eseguiAssolo() {
      //Scala maggiore in DO
      System.out.println("DO RE MI FA SOL LA SI");
   }
}
```

Interfacce

In questo modo è possibile ereditare questo metodo in un'eventuale sottoclasse senza dover riscrivere il metodo. La seguente classe compila senza problemi:

```
package musica;

public class Musicista implements Solista {

}
```

Interfacce

Dalla versione 9 di Java è possibile creare anche metodi privati ausiliari nelle interfacce. I metodi privati possono essere invocati solo dai metodi di default dichiarati nella stessa interfaccia.

Interfacce

Non bisogna dichiararli come metodi di default, appunto perché sono solo da intendere come metodi ausiliari e non vengono ereditati.

Interfacce

Interfacce funzionali

Per definizione prendono il nome di **interfacce funzionali** le interfacce che contengono un unico metodo astratto (SAM, che sta per Single Abstract Method).

Interfacce

Vengono chiamate così proprio perché è come se esistessero solo per dichiarare una funzione da implementare.

Interfacce

Esiste anche un nuovo package che dichiara una serie di interfacce funzionali: il package java.lang.function.

Interfacce

Ereditarietà multipla

Era possibile implementare più interfacce anche prima di Java 8 ma si ereditavano metodi astratti che bisognava riscrivere.

Interfacce

Non è possibile istanziare né classi astratte né interfacce. Inoltre il vantaggio comune che offrono sia le classi astratte sia le interfacce risiede nel fatto che esse possono obbligare le sottoclassi ad implementare comportamenti.

Interfacce

Si consideri il seguente esempio, la classe veicolo.

```
package veicolo;

public abstract class Veicolo {

    public abstract void accelera();
    public abstract void decelera();

}
```

Interfacce

Per esempio, ogni veicolo accelera e decelera, quindi tutte le classi non astratte che estendono Veicolo devono riscrivere i metodi accelera() e decelera().

Interfacce

```
package veicolo;

public class Aereo extends Veicolo {
   // override del metodo ereditato
   public void accelera() {
      // ...
   }
   // override del metodo ereditato
   public void decelera() {
      // ...
       }

   // ...
}
```

Interfacce

Considerando sempre l'esempio del Veicolo superclasse di Aereo, potremmo introdurre un'interfaccia Volante:

```
package veicolo;

public interface Volante {

   void atterra();
   void decolla();

}
```

Ogni classe che deve astrarre un concetto di oggetto volante deve implementare l'interfaccia.

Interfacce

Riscrivendo la classe:

```
package veicolo;

public class Aereo extends Veicolo implements Volante {
   public void accelera() {    // override del metodo ereditato
      // ...
   }
   public void decelera() {    // override del metodo ereditato
      // ...
   }
   public void atterra() {     // override del metodo di Volante
      // ...
   }
   public void decolla() {     // override del metodo di Volante
      // ...
   }

   // ...
}
```

Domande e approfondimenti